

SEC102

MENACES INFORMATIQUES ET CODES MALVEILLANTS
ANALYSE ET LUTTE

Calendrier prévisionnel de la formation

Cours	Date	Titre du cours	Enseignant
01	mar. 07/03/2023 (sem. 10)	Présentation du cours SEC 102 avec le fil rouge : la gestion de l'incident de sécurité en entreprise, Introduction à la gestion de l'incident de sécurité en entreprise	Fabrice CRASNIER
02	mer. 15/03/2023 (sem. 11)	Le métier de pentester et ses contraintes	Anthony DESSIATNIKOFF
03	mer. 22/03/2023 (sem. 12)	Les méthodes d'intrusion sur les systèmes, méthodologie du hacker pour créer une intrusion (en VM)	Anthony DESSIATNIKOFF
04	mar. 28/03/2023 (sem. 13)	Posture de la DSI face à un incident de sécurité. Présentation de la prestation de réponse sur incident, gestion de crise (mise en place des outils de crise).	Fabrice CRASNIER
05	mer 05/04/2023 (sem 14)	De l'investigation sur le réseau (ingénieur réseaux) à la collecte des données, Exploitation et analyse des journaux réseaux.	Félix GUYARD
06	mer 12/04/2023 (sem 15) Distanciel	De l'investigation sur le système (ingénieur système) à la collecte des données. Exploitation et analyse des données syslog/sysmon	Félix GUYARD
07	mar. 18/04/2023 (sem.16)	Analyse de malwares	Anthony DESSIATNIKOFF

3

Calendrier prévisionnel de la formation

Cours	Date	Titre du cours	Enseignant
08	mer 10/05/2023 (sem 19)	Analyse mémoire.	Félix GUYARD
09	lun 15/05/2023 (sem 20)	L'analyse forensic post mortem	Fabrice CRASNIER
10	mer 23/05/2023 (sem 21)	Développer un plan de remédiation après un incident de sécurité.	Félix GUYARD
11	Mar. 30/05/2023 (sem 22)	Rédaction du rapport de réponse sur incident, la conservation des preuves,	Fabrice CRASNIER
12	lun. 05/06/2023 (sem.23)	Présentation des audits de sécurité	Anthony DESSIATNIKOFF
13	lun. 12/06/2023 (sem.24)	Introduction au langage Python appliqué à cybersécurité	Anthony DESSIATNIKOFF
14	lun. 19/06/2023 (sem.25)	RGPD, préparation d'un dossier de plainte, poursuite vers une juridiction	Fabrice CRASNIER

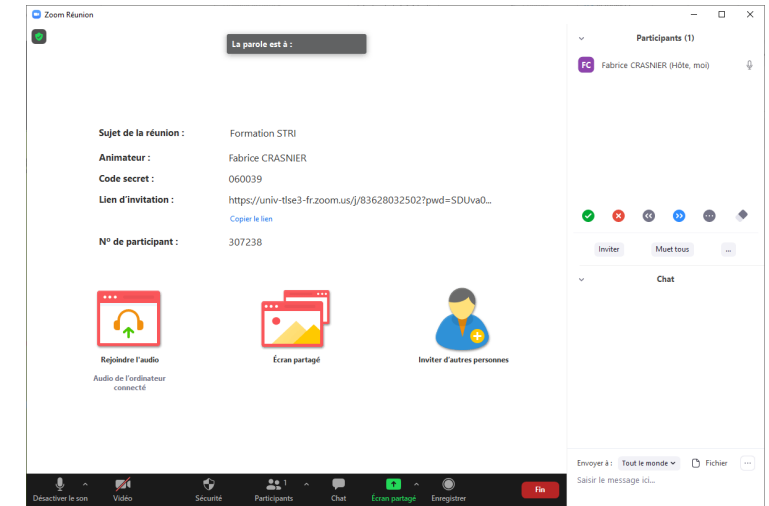
4

Bienvenue sur la formation en ligne via Zoom

Coordonnées de la salle ZOOM

Fabrice CRASNIER vous invite à une réunion Zoom planifiée.

Sujet : Cours CNAM SEC102



Participer à la réunion Zoom

<https://univ-tlse3-fr.zoom.us/j/91611860094?pwd=K3YwNHbZb3UyYmJMbM9oTXBRdy9WZz09>

ID de réunion : 916 1186 0094

Code secret : 687038



Bienvenue sur la formation en ligne via Zoom

5

Affichage de la zone de
Participants

The screenshot shows a Zoom meeting window titled "Zoom Réunion". The main area displays the meeting details: "La parole est à : [barre]", "Formation STRI", "Fabrice CRASNIER", "Code secret : 060039", "Lien d'invitation : https://univ-tlse3-fr.zoom.us/j/83628032502?pwd=SDUva0...", "Copier le lien", and "N° de participant : 307238". Below this are three icons: "Rejoindre l'audio" (Audio de l'ordinateur connecté), "Écran partagé", and "Inviter d'autres personnes". On the right, a "Participants (1)" sidebar shows "FC Fabrice CRASNIER (Hôte, moi)". Below the participants is a toolbar with icons for mute, video, chat, and screen sharing, along with buttons for "Inviter", "Muet tous", and "...". At the bottom, a "Chat" sidebar is visible with a text input field "Saisir le message ici..." and a "Fin" button. A blue arrow points from the "Affichage de la zone de Participants" text to the participants sidebar. A yellow arrow points from the "QCM" box to the toolbar. A green arrow points from the "Affichage de la zone du Chat" text to the chat sidebar.

Zoom Réunion

La parole est à :

Formation STRI

Fabrice CRASNIER

Code secret : 060039

Lien d'invitation : <https://univ-tlse3-fr.zoom.us/j/83628032502?pwd=SDUva0...>

[Copier le lien](#)

N° de participant : 307238

Rejoindre l'audio

Audio de l'ordinateur connecté

Écran partagé

Inviter d'autres personnes

Participants (1)

FC Fabrice CRASNIER (Hôte, moi)

Inviter Muet tous ...

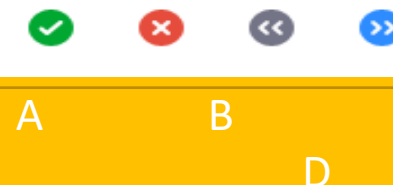
Chat

Envoyer à : Tout le monde Fichier ...

Saisir le message ici...

Fin

QCM



Affichage de la zone du
Chat

13 - lun. 12/06/2023 (sem. 24)

MENACES INFORMATIQUES ET CODES MALVEILLANTS ANALYSE ET LUTTE

Introduction au langage Python
appliqué à cybersécurité

Sommaire

- **Introduction au langage Python**
- **Bases du langage Python**
- **Bibliothèques Python utiles pour la cybersécurité**
 - **Requests**
 - **Scapy**
 - **Pwntools**
 - **Pycryptodome et hashlib**
- **Conclusion**

- Les bases du langage
- Syntaxe et structure
- Types de données

Introduction langage Python



Introduction

Langage de script (pas de compilation nécessaire)

Python version 3 (non compatible avec version 2)

Dernière version stable (juin 2023) : 3.11

Attention à l'indentation !

Ne pas nommer son fichier .py comme un nom de bibliothèque existant !



Pourquoi utiliser Python ?

- Créé en 1991 (suffisamment testé depuis)
- Simple et efficace avec peu de lignes de code
- Documentation très complète
- **Beaucoup** de bibliothèques disponibles pour tout faire
- Interfaces possibles entre plusieurs langages : C (Cython) / Java (Jython)

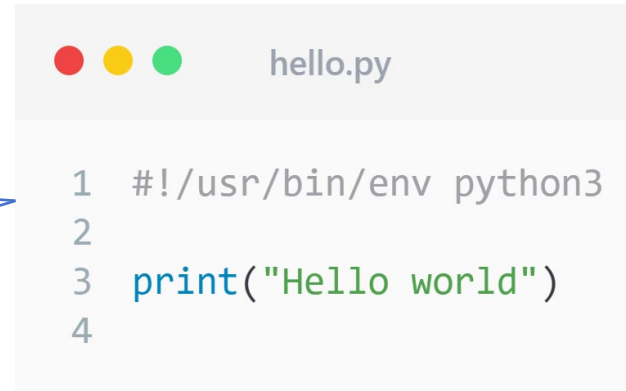
11

Introduction : exécution dans un fichier (script)

2 possibilités d'exécution d'un script

- Ligne 1 non nécessaire :
\$ python hello.py
Hello world

- Ligne 1 nécessaire :
\$ chmod +x hello.py
\$./hello.py
Hello world



```
1 #!/usr/bin/env python3
2
3 print("Hello world")
4
```


Introduction : exécution en live

On peut aussi utiliser l'interpréteur Python :

```
$ python
Python 3.11.2 (main, Mar 13 2023, 12:18:29) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

← Ecrire le code Python ici ligne par ligne

Ecriture : recommandations du Guide de style (PEP 8)

<https://peps.python.org/pep-0008/>

Indentation : 4 espaces / niveau

Nommage de fonctions et variables : minuscules avec «_»

Nommage des constantes : majuscules avec «_»

Installation de bibliothèques

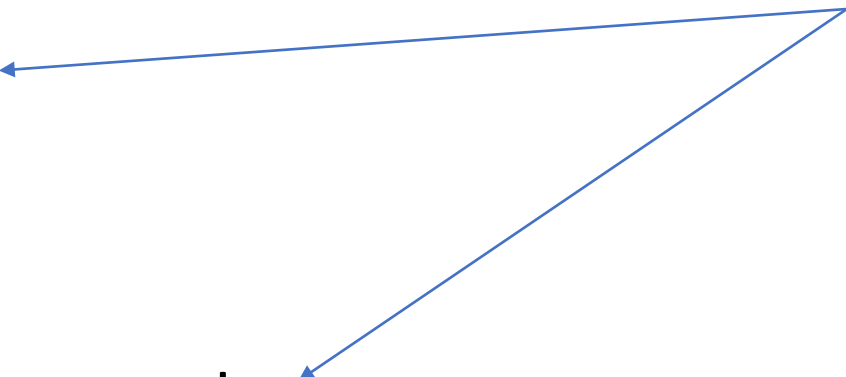
Par exemple pour installer requests :

```
$ pip install requests
```

ou

```
$ python -m pip install requests
```

Pour avoir pip :
apt-get install pip-python



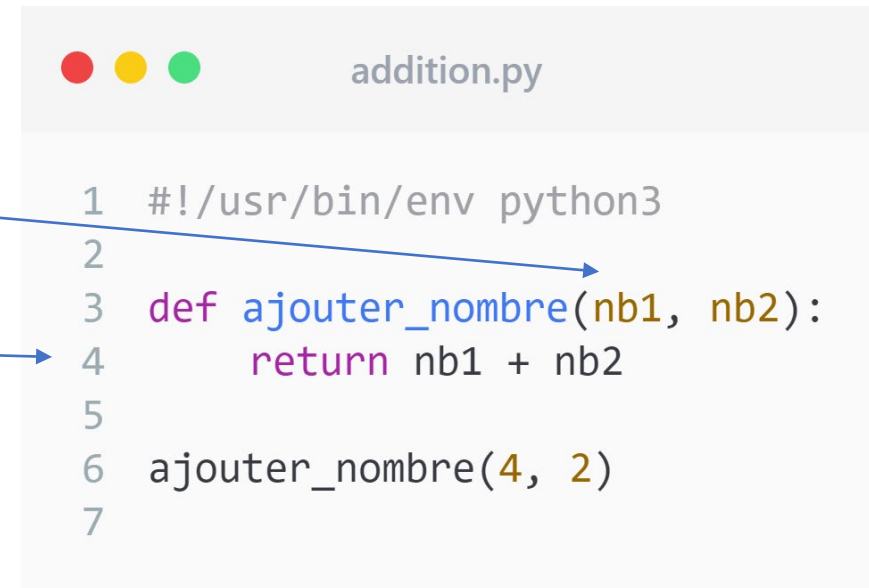
15 Les bases : les fonctions

On utilise le mot-clé : « def » avant le nom

Ici, 2 paramètres : nb1 et nb2

Et on retourne un résultat

=> Ensuite, il faut appeler cette fonction !



```
1  #!/usr/bin/env python3
2
3  def ajouter_nombre(nb1, nb2):
4      return nb1 + nb2
5
6  ajouter_nombre(4, 2)
7
```

The image shows a code editor window titled 'addition.py'. The code is written in Python and includes a shebang line, a function definition, and a function call. Two blue arrows originate from the text on the left: one points from '« def »' to the 'def' keyword on line 3, and another points from 'un résultat' to the 'return' statement on line 4.

16

Les bases : bien gérer les types de données

Il faut prendre en comptes les types.

On ajoute des entiers avec des entiers, on concatène uniquement des chaines de caractères, etc.

Ex : on n'ajoute pas un entier avec une chaine de caractères !

int + str = ✗

str + str = ✓

```
>>> print(5 + 'e')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> print(str(5) + 'e')
5e
>>>
```

Différents types de données

Les nombres entiers : 2 8 3443

Les nombres flottants : 2,5 6,41254 3,14

Les chaînes de caractères : "a" "voiture" "hello"

Les booléens : True ou False (Majuscule obligatoire)

Les listes : [1, "a", 7, 45]

Les dictionnaires : {1:5, 9:"a"}

Les chaînes de caractères

Il faut utiliser des guillemets simples ou doubles :

'a' ou "a"

"voiture"

"hello"

On peut les concaténer :

```
"hello" + "world"  
>>> "helloworld"
```

```
a = "bbb"  
b = "ccc"  
>>> a + ' ' + b = "bbb ccc"
```

Les nombres entiers

On peut utiliser 3 notations différentes pour un même nombre :

136524 (décimal)

0x2154c (hexadécimal)

0b100001010101001100 (binaire)

Différents types de données : les listes [...]

Ce qu'on appelle souvent des tableaux dans les autres langages.

Une liste peut contenir tous les types.

Elle est ordonnée.

Une chaîne de caractères est une liste donc même opérations possibles



les_listes.py

```
1
2 liste = [1, 2, 3, 4, 5]
3
4 print(liste[0]) # [1]
5 print(liste[1:3]) # [2, 3, 4]
6 print(liste[-1]) # [5]
7
```


Différents types de données : les dictionnaires {...}

Les dictionnaires : dico = {1:5, "9":"a"}

Différent des listes, ils permettent de créer des références
clé:valeur

On peut accéder facilement à une valeur grâce à sa clé :

dico["9"] # renverra "a"

Les commentaires

2 types de commentaires :

- inline
- multiline

comments.py

```
1 # inline
2
3 """
4 multiline 1
5 multiline 2
6 multiline 3
7 """
```

Programmation orientée objet (si besoin)

Possible de créer des classes / méthodes / propriétés.

Constructeur

Propriétés de la classe

Méthode de classe

Respecter l'indentation !



les_classes.py

```
1 class Voiture:
2
3     def __init__(self):
4         self.nb_roues = 4
5         self.proprietaire = "Jose Nailloux"
6         self.vitesse = 0
7
8     def avancer(self):
9         print(f"Je suis rapide : {self.vitesse}")
10
11 v = Voiture()
12 v.avancer()
```

Conditions (if) et boucles (for)

- Condition simple (doit renvoyer un booléen)
- Boucle avec for (« while » existe aussi)

```
les_conditions.py

1
2  if valeur == True:
3      print("vrai")
4  else:
5      print("faux")
6
7  liste = [1, 3, 8, 4, 2]
8  for i in liste:
9      print(i)
```

Import de modules

- On peut importer des classes, fichiers :

`import test`

ou

`from test import *`

Le fichier « test.py » sera importé
et toutes ses fonctions

`test.function()` pour appeler une fonction

Les fonctions built-in

- Certaines fonctions existent déjà dans Python telles que :

```
chr(125) # }  
ord('}') # 125  
hex(125) # 0x7d  
bin(125) # 0b1111101  
str(125) # "125"
```


Le main()

- Pour mieux identifier le début du script, commencer par :

```
utiliser_main.py

1  #!/usr/bin/env python3
2
3  """
4  Avant, on définit les fonctions
5  """
6
7  if __name__ == "__main__":
8      # debut du script ici
9
```

- os
- sys
- time

Bibliothèques Python utiles



import os

- <https://docs.python.org/3/library/os.html>
- Réaliser des actions sur le système

```
utiliser_os.py

1  #!/usr/bin/env python3
2
3  import os
4
5  os.chdir("/home/user/path")
6  print(os.getcwd()) # "/home/user/path"
7  print(os.getpid()) # <pid>
8
9  os.system("id")
10
```

import sys

- <https://docs.python.org/3/library/sys.html>
- Ex : pour récupérer les arguments (sys.argv) de la ligne de commande :

```
python utilisier_sys.py hello
```

```
utilisier_sys.py  
hello
```

```
utilisier_sys.py  
  
1  #!/usr/bin/env python3  
2  
3  import sys  
4  
5  print(sys.argv[0])  
6  print(sys.argv[1])
```

31

import time

- <https://docs.python.org/3/library/time.html>
- Récupérer l'heure du système
- Calcul sur des dates, timestamp
- Suspend l'exécution avec `time.sleep(sec)`

- Requests
- Scapy
- Pwntools
- Pycryptodome et hashlib

Bibliothèques Python utiles pour la cybersécurité



La bibliothèque Requests

- <https://requests.readthedocs.io/en/latest/>
- Permet d'effectuer des requêtes HTTP simplement.
- Ici, une requête HTTP GET vers `www.google.com`
- Puis on affiche le contenu HTML résultat.

```
utiliser_requests.py

1  #!/usr/bin/env python3
2
3  import requests
4
5  url = "https://www.google.com"
6  res = requests.get(url)
7
8  print(res.content)
9
```


La bibliothèque Requests

- Pour une méthode POST :

```
utiliser_requests2.py

1  #!/usr/bin/env python3
2
3  import requests
4
5  url = "https://www.example.com/auth.php"
6  data = {'username': 'admin', 'password': 'PASSWD'}
7  res = requests.post(url, data=data)
8
9  print(res.content)
10
```

La bibliothèque Requests

- Passer par un proxy
- Utiliser un dictionnaire qui contient les clés http et https

```
utiliser_requests3.py

1  #!/usr/bin/env python3
2
3  import requests
4
5  proxy = {'http': 'http://127.0.0.1:8080'}
6  url = "https://www.example.com/auth.php"
7  data = {'username': 'admin', 'password': 'PASSWD'}
8  res = requests.post(url, data=data, proxies=proxy)
9
10 print(res.content)
11
```

Scapy

- <https://scapy.readthedocs.io/en/latest/>
- Bibliothèque très connue de manipulation de paquets réseaux
- Scapy is a Python program that enables the user to send, sniff and dissect and forge network packets. This capability allows construction of tools that can probe, scan or attack networks.

- Création de paquets réseaux :

```
1  #!/usr/bin/env python3
2
3  from scapy import *
4
5  a = IP(dst="172.16.1.40")
6
7  b = Ether(dst="ff:ff:ff:ff:ff:ff") / IP(dst=["ketchup.com", "mayo.com"], ttl=(1,9)) / UDP()
8
9  c = IP(dst=["ketchup.com", "mayo.com"], ttl=(1,9)) / TCP()
10
```

- D'autres exemples :

```
utiliser_scapy2.py

1  #!/usr/bin/env python3
2
3  from scapy import *
4
5  # lire un fichier pcap
6  a = rdpcap("/spare/captures/isakmp.cap")
7
8  # ecrire un fichier pcap
9  wrpcap("temp.cap",pkts)
10
11 # envoyer requete HTTP GET
12 b = Ether()/IP(dst="www.slashdot.org")/TCP()/"GET /index.html HTTP/1.0 \n\n"
13
14 # sniffer 2 paquets ICMP avec l'adresse IP 66.35.250.151
15 sniff(filter="icmp and host 66.35.250.151", count=2)
16
```

Pwntools

- <https://docs.pwntools.com/en/stable/>
- Un framework de CTF et une bibliothèque de développement de code d'exploitation
- `pip install pwntools`

Pwntools

- Connexion à un port
- Envoi / réception de données

```
utiliser_pwntools.py

1  #!/usr/bin/env python3
2
3  from pwn import *
4
5  conn = remote('ftp.ubuntu.com',21)
6
7  conn.recvline()
8
9  conn.send(b'USER anonymous\r\n')
10
11 conn.recvuntil(b' ', drop=True)
12
13 conn.recvline()
14
15 conn.close()
```

Cryptographie : Pycryptodome

- <https://www.pycryptodome.org/>
- Permet de réaliser du chiffrement symétrique / asymétrique
- `pip install pycryptodome`

42 Cryptographie : Hashlib

- <https://docs.python.org/3/library/hashlib.html>
- Permet de réaliser des hashes : md5, sha1, sha256, sha384, sha512

utiliser_hashlib.py

```
1  #!/usr/bin/env python3
2
3  import hashlib
4
5  m = hashlib.sha256()
6  m.update(b"Nobody inspects")
7  m.update(b" the spammish repetition")
8
9  print(m.digest()) # b'\x03\x1e\xdd}Ae\x15\x93\xc5\xfe\\\x00o\xa5u+7\xfd\xdf\xf7\xbcN\x84:\xa6\xaf\x0c\x95\x0fK\x94\x06'
10 print(m.hexdigest()) # '031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b9406'
11
```

Pycryptodome : chiffrement symétrique

- Utiliser AES 128 bits, 256 bits
- Plusieurs modes disponibles (CBC, CCM, CFB, CTR, GCM, OFB)

```
utiliser_crypto.py

1  #!/usr/bin/env python3
2
3  from Crypto.Cipher import AES
4  from Crypto.Random import get_random_bytes
5
6  data = b'secret data'
7
8  key = get_random_bytes(16)
9  cipher = AES.new(key, AES.MODE_EAX)
10 ciphertext, tag = cipher.encrypt_and_digest(data)
11
12 file_out = open("encrypted.bin", "wb")
13 [ file_out.write(x) for x in (cipher.nonce, tag, ciphertext) ]
14 file_out.close()
```

Pycryptodome : chiffrement asymétrique

- Utiliser RSA 2048 bits



utiliser_crypto2.py

```
1  #!/usr/bin/env python3
2
3  from Crypto.PublicKey import RSA
4
5  secret_code = "Unguessable"
6  key = RSA.generate(2048)
7  encrypted_key = key.export_key(passphrase=secret_code, pkcs=8,
8                                protection="scryptAndAES128-CBC")
9
10 file_out = open("rsa_key.bin", "wb")
11 file_out.write(encrypted_key)
12 file_out.close()
13
14 print(key.publickey().export_key())
```

Pycryptodome : générateur aléatoire sécurisé

- `get_random_bytes(n)`
- `randint(a, b)`
- `getrandbits(n)`

```
utiliser_crypto3.py

1  #!/usr/bin/env python3
2
3  from Crypto.PublicKey import RSA
4  from Crypto.Random import get_random_bytes
5  from Crypto.Cipher import AES, PKCS1_OAEP
6
7  data = "I met aliens in UFO. Here is the map.".encode("utf-8")
8  file_out = open("encrypted_data.bin", "wb")
9
10 recipient_key = RSA.import_key(open("receiver.pem").read())
11 session_key = get_random_bytes(16)
12
13 # Encrypt the session key with the public RSA key
14 cipher_rsa = PKCS1_OAEP.new(recipient_key)
15 enc_session_key = cipher_rsa.encrypt(session_key)
16
17 # Encrypt the data with the AES session key
18 cipher_aes = AES.new(session_key, AES.MODE_EAX)
19 ciphertext, tag = cipher_aes.encrypt_and_digest(data)
20 [ file_out.write(x) for x in (enc_session_key, cipher_aes.nonce, tag, ciphertext) ]
```



46

Conclusion

- Les bases de Python
- Quelques bibliothèques